

CONTRACTIQ

Contract Obligation Tracking & Compliance Management Platform

SPRINT 3 – DATABASE DESIGN

Database Schema, Relationships and Entity Relationship Diagram

Design activity only — no API implementation required for this sprint.

1. Objective

The objective of Sprint 3 is to design the database structure required for the ContractIQ platform. The design identifies the core tables, important columns, primary keys, foreign keys and relationships that provide the foundation for later backend implementation.

2. Database Tables Overview

Table	Purpose
users	Stores users who access and operate the ContractIQ platform.
contracts	Stores the main contract records managed by the platform.
contract_versions	Maintains the history of different versions of a contract.
obligations	Tracks contractual obligations, their deadlines, status, priority, and ownership.
renewals	Tracks renewal events and renewal-related information for contracts.
notifications	Stores alerts and notifications delivered to users about contracts and related events.
reports	Stores generated reports associated with contracts and users.
audit_logs	Provides an audit trail of changes and actions for compliance and traceability.
activities	Tracks user/system activities related to contracts and the platform.

3.1 users

Stores users who access and operate the ContractIQ platform.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique user identifier
full_name	VARCHAR(100)	—	User's full name
email	VARCHAR(255)	UNIQUE	User email address
password_hash	VARCHAR(255)	—	Stored password hash
role	VARCHAR(50)	—	Application role
is_active	BOOLEAN	—	Whether the account is active
created_at	TIMESTAMP	—	Record creation time
updated_at	TIMESTAMP	—	Last update time

3.2 contracts

Stores the main contract records managed by the platform.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique contract identifier
title	VARCHAR(255)	—	Contract title
contract_number	VARCHAR(100)	UNIQUE	Business contract reference
description	TEXT	—	Contract description
party_name	VARCHAR(255)	—	Other contract party
start_date	DATE	—	Contract start date
end_date	DATE	—	Contract end date
status	VARCHAR(50)	—	Current contract status
owner_id	INTEGER	FK → users.id	User responsible for the contract
created_at	TIMESTAMP	—	Record creation time
updated_at	TIMESTAMP	—	Last update time

3.3 contract_versions

Maintains the history of different versions of a contract.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique version identifier
contract_id	INTEGER	FK → contracts.id	Contract to which the version belongs
version_number	INTEGER	—	Version sequence number
document_path	VARCHAR(500)	—	Location of the version document
change_summary	TEXT	—	Summary of changes in the version
created_by	INTEGER	FK → users.id	User who created the version
created_at	TIMESTAMP	—	Version creation time

3.4 obligations

Tracks contractual obligations, their deadlines, status, priority, and ownership.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique obligation identifier
contract_id	INTEGER	FK → contracts.id	Contract containing the obligation
title	VARCHAR(255)	—	Obligation title
description	TEXT	—	Obligation details
due_date	DATE	—	Obligation due date
status	VARCHAR(50)	—	Current obligation status
priority	VARCHAR(50)	—	Obligation priority
assigned_to	INTEGER	FK → users.id	User responsible for the obligation
created_at	TIMESTAMP	—	Record creation time
updated_at	TIMESTAMP	—	Last update time

3.5 renewals

Tracks renewal events and renewal-related information for contracts.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique renewal identifier
contract_id	INTEGER	FK → contracts.id	Contract being renewed
renewal_date	DATE	—	Renewal date
status	VARCHAR(50)	—	Renewal status
renewal_period	INTEGER	—	Renewal period
notes	TEXT	—	Additional renewal notes
created_at	TIMESTAMP	—	Record creation time

3.6 notifications

Stores alerts and notifications delivered to users about contracts and related events.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique notification identifier
user_id	INTEGER	FK → users.id	Recipient user
contract_id	INTEGER	FK → contracts.id	Related contract, when applicable
title	VARCHAR(255)	—	Notification title
message	TEXT	—	Notification content
notification_type	VARCHAR(50)	—	Type of notification
is_read	BOOLEAN	—	Whether the notification has been read
created_at	TIMESTAMP	—	Notification creation time

3.7 reports

Stores generated reports associated with contracts and users.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique report identifier
contract_id	INTEGER	FK → contracts.id	Contract covered by the report
generated_by	INTEGER	FK → users.id	User who generated the report
report_type	VARCHAR(100)	—	Type of report
report_path	VARCHAR(500)	—	Location of the generated report
created_at	TIMESTAMP	—	Report creation time

3.8 audit_logs

Provides an audit trail of changes and actions for compliance and traceability.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique audit log identifier
user_id	INTEGER	FK → users.id	User associated with the action
contract_id	INTEGER	FK → contracts.id	Related contract, when applicable
action	VARCHAR(100)	—	Action performed
entity_type	VARCHAR(100)	—	Type of entity affected
entity_id	INTEGER	—	Identifier of the affected entity
old_value	JSON	—	Previous value/state
new_value	JSON	—	New value/state
created_at	TIMESTAMP	—	Audit event time

3.9 activities

Tracks user/system activities related to contracts and the platform.

Column	Data Type	Key / Constraint	Description
id	INTEGER	PK	Unique activity identifier
user_id	INTEGER	FK → users.id	User associated with the activity
contract_id	INTEGER	FK → contracts.id	Related contract, when applicable
activity_type	VARCHAR(100)	—	Type of activity
description	TEXT	—	Activity details
created_at	TIMESTAMP	—	Activity creation time

4. Primary Keys and Foreign Keys

Each table has a primary key named id. Foreign keys connect child records to their parent records.

Table	Foreign Key	References
contracts	owner_id	users.id
contract_versions	contract_id	contracts.id
contract_versions	created_by	users.id
obligations	contract_id	contracts.id
obligations	assigned_to	users.id
renewals	contract_id	contracts.id
notifications	user_id	users.id
notifications	contract_id	contracts.id
reports	contract_id	contracts.id
reports	generated_by	users.id
audit_logs	user_id	users.id
audit_logs	contract_id	contracts.id
activities	user_id	users.id
activities	contract_id	contracts.id

5. Table Relationships

Parent	Child	Relationship	Explanation
users	contracts	One-to-Many	One user can own/manage many contracts.
users	contract_versions	One-to-Many	One user can create many contract versions.
contracts	contract_versions	One-to-Many	One contract can have many versions.
users	obligations	One-to-Many	One user can be assigned many obligations.
contracts	obligations	One-to-Many	One contract can contain many obligations.
contracts	renewals	One-to-Many	A contract can have multiple renewal records over time.
users	notifications	One-to-Many	One user can receive many notifications.
contracts	notifications	One-to-Many	A contract can be associated with many notifications.
users	reports	One-to-Many	One user can generate many reports.
contracts	reports	One-to-Many	One contract can have many reports.
users	audit_logs	One-to-Many	One user can generate many audit events.
contracts	audit_logs	One-to-Many	A contract can have many audit events.
users	activities	One-to-Many	One user can have many recorded activities.
contracts	activities	One-to-Many	A contract can have many recorded activities.

6. Entity Relationship Diagram

The ER diagram below represents the nine required tables and their relationships. It was generated from the DBML design supplied for the Sprint 3 database planning activity.

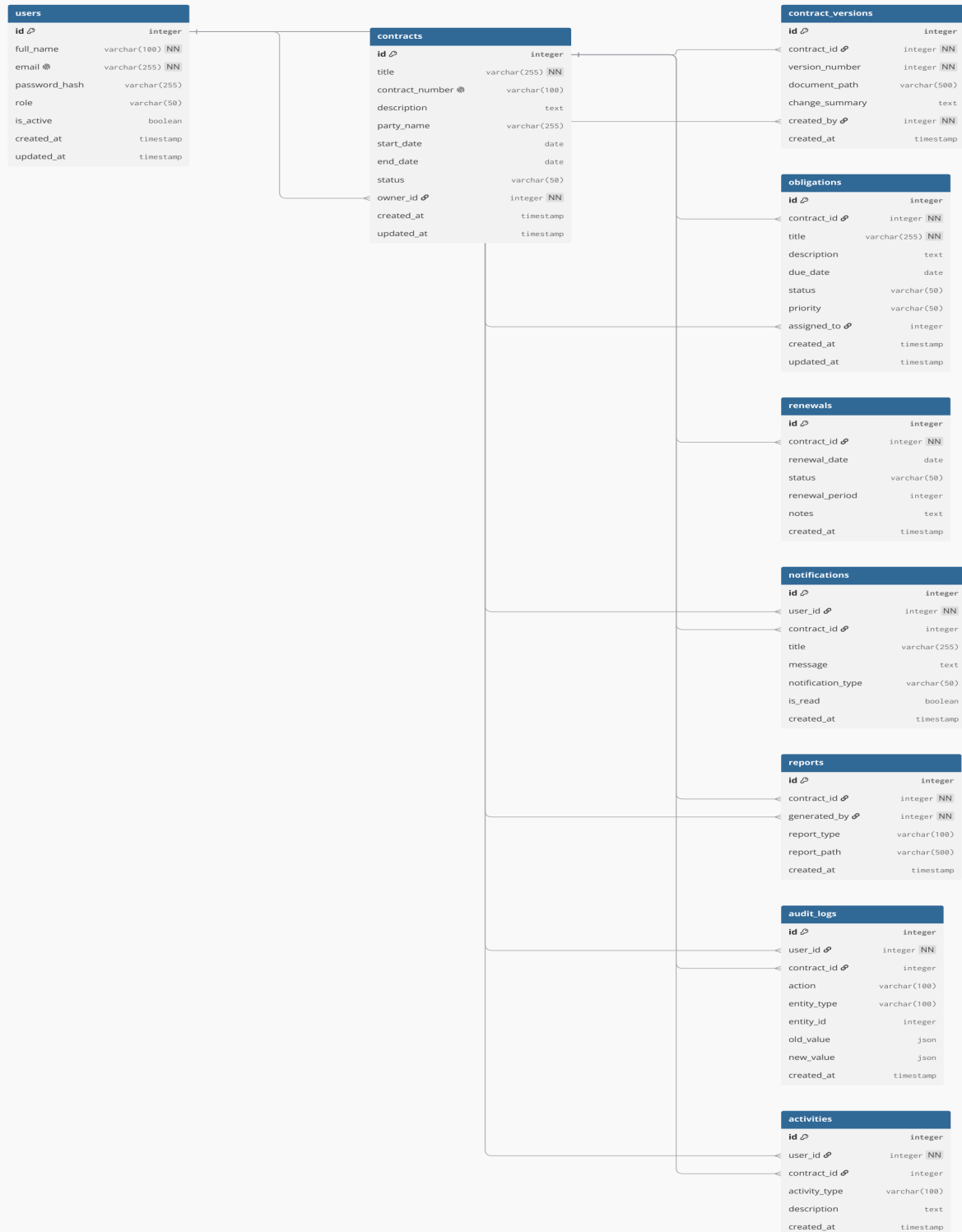


Figure 1. ContractIQ database ER diagram.

7. How the Tables Work Together

The users and contracts tables form the central part of the design. A user can own or manage multiple contracts, while each contract can have multiple versions, obligations and renewal records. Contract versions preserve document history, obligations track responsibilities and deadlines, and renewals capture renewal events.

Notifications connect users and contracts so relevant alerts can be recorded. Reports store generated contract-related reports. Audit logs provide traceability by recording actions and before/after values, while activities provide a record of user or system activity associated with contracts. Together these relationships provide the database foundation required for contract tracking and compliance management.

8. Conclusion

This Sprint 3 database design provides a structured foundation for ContractIQ. The identified tables and relationships cover user management, contract lifecycle tracking, obligations, renewals, notifications, reporting, auditing and activity tracking. The design can be used as the reference for implementing the remaining SQLAlchemy models and backend modules in the next development phase.